Báo cáo phương pháp phát hiện và giải mã QR Code

Sinh viên: Đinh Văn Sinh

MSSV: 22022615

Tóm tắt nội dung

Báo cáo trình bày hệ thống phát hiện và định vị QR code hoàn toàn dựa trên các kỹ thuật xử lý ảnh cổ điển (OpenCV, NumPy), không sử dụng thư viện giải mã chuyên dụng (pyzbar, zxing) hay mô hình deep learning. Phương pháp dựa trên đặc trưng ba finder pattern hình vuông lồng nhau của QR code, kết hợp nhiều cơ chế dự phòng cho ảnh mờ, méo, nhiều QR nhỏ.

1. Mô tả phương pháp

Hệ thống gồm bốn bước chính: tiền xử lý ảnh, phát hiện finder pattern, nhóm ba pattern thành QR và tinh chỉnh bounding box axis-aligned, cuối cùng là giải mã nội dung.

1.1. Tiền xử lý ảnh

Ảnh đầu vào được chuyển sang grayscale, áp dụng gamma correction nếu ảnh quá tối (giá trị trung bình < 90). Bộ lọc bilateral (d=5, sigmaColor=35, sigmaSpace=35) làm mịn nhưng giữ cạnh. CLAHE (clipLimit=3.0, tileGridSize=8×8) tăng cường độ tương phản cục bộ. Sau đó, ảnh được nhị phân hóa bằng tổ hợp Otsu và adaptive threshold (Gaussian, block size phụ thuộc kích thước ảnh). Morphological closing (kernel 3×3) được dùng để nối các module bị đứt, cuối cùng loại bỏ thành phần có diện tích nhỏ (minArea = max(300, 0,01% diện tích ảnh)).

1.2.Phát hiện finder pattern

Sử dụng cv2.findContours với chế độ RETR_TREE để lấy cây phân cấp. Duyệt các contour ngoài cùng, tìm chuỗi ba contour lồng nhau (ngoài – giữa – trong). Với mỗi bộ ba, kiểm tra:

• Mỗi contour xấp xỉ hình vuông (4–8 cạnh, tỷ lệ cạnh 2,8, độ đặc 0,5).

• Diện tích giảm dần và tỷ lệ diện tích nằm trong khoảng (1,15, 6,5) cho lớp ngoài/giữa và (1,15, 8,5) cho lớp giữa/trong.

• Ba tâm gần nhau (khoảng cách 35% kích thước contour ngoài).

Kết quả là danh sách các finder pattern (contour ngoài, tâm, diện tích).

1.3. Nhóm ba pattern thành QR

Với mỗi bộ ba pattern, xác định cạnh dài nhất → hai điểm còn lại là top-right và bottom-left, điểm còn lại là top-left. Kiểm tra điều kiện tam giác vuông cân (cos góc 0,45 và tỷ lệ cạnh 0,42). Lấy ba đỉnh tương ứng từ contour của từng pattern (dùng minAreaRect hoặc xấp xỉ đa giác, chọn đỉnh dựa trên tổng và hiệu tọa độ). Góc thứ tư được tính bằng phép biến đổi affine từ ba tâm pattern sang hình vuông đơn vị và ánh xạ ngược. Tứ giác kết quả được sắp xếp theo chiều kim đồng hồ, bắt đầu từ top-left.

1.4.Tinh chỉnh bounding box axis-aligned

Do ground truth là hình chữ nhật thẳng trục, tứ giác tìm được được chuyển về axis-aligned bằng cách:

1. Cắt vùng mở rộng (padding 25% kích thước) quanh rough quad.
2. Nhị phân hóa vùng (Otsu + adaptive), đóng morphology.
3. Tìm contour ngoài lớn nhất, lấy bounding rectangle.
4. Quét dòng/cột (max_expand=30) để tinh chỉnh biên dựa trên tỷ lệ điểm đen 8%.
5. Cắt bỏ viền thừa bằng projection profile (giữ các dòng/cột có tỷ lệ đen > 8%).

Kết quả là hình chữ nhật (x1,y1,x2,y2).

1.5.Giải mã nội dung

Sau khi có bounding box, vùng ảnh được mở rộng (pad 12% kích thước nhỏ nhất, tối thiểu 4 pixel, tối đa 44 pixel) rồi giải mã bằng cv2.QRCodeDetector.detectAndDecode(). Nếu thất bại, thử với ảnh upscale 2×. Kết quả được chuẩn hóa UTF-8.

2 Kỹ thuật và thư viện sử dụng

2.1 Kỹ thuật

Gamma correction, bilateral filter, CLAHE, Otsu + adaptive threshold, morphological operations, phân tích cây phân cấp contour, hình học tam giác vuông cân, affine transformation, connected components, texture analysis (transition score), NMS dựa trên IoU, quét projection profile.

2.2 Thư viện

OpenCV, NumPy, Python threading, argparse, csv, os, time. Không sử dụng pyzbar, zxing, hay bất kỳ mô hình deep learning nào cho phát hiện.

3. Kết quả trên tập public

3.1. Độ chính xác

Trên tập train khi chạy: python main.py –data public_train.csv

Chỉ số	Số lượng QRcode	Định vị(IoU >= 0.5)
Recall	72.26%	54.42%
Precision	95.33%	71.80%
F1 Score	82.21%	61.91%

Trên tập valid khi chạy: python main.py –data public_valid.csv

Chỉ số	Số lượng QRcode	Định vị(IoU >= 0.5)
Recall	72.26%	66.73%
Precision	96.35%	77.40%
F1 Score	89.22%	71.67%

3.2. Tốc độ xử lý

Cấu hình: CPU, RAM 16 GB, không GPU.

Trên tập valid:

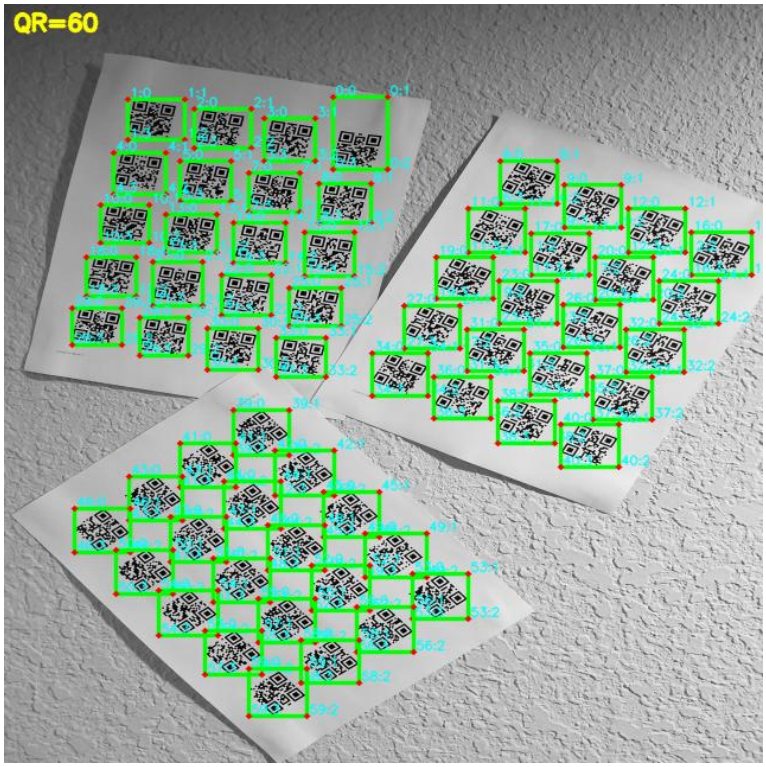
- Tổng thời gian chạy : 149.23 giây
- Tốc độ trung bình: 0.4829 giây/ảnh

Trên tập train:

- Tổng thời gian chạy : 581.23 giây
- Tốc độ trung bình: 0.5367 giây/ảnh

3.3. Các trường hợp đúng/sai tiêu biểu

Đúng (TP): QR rõ nét, xoay 45°, nhiều QR nhỏ xếp hàng, ảnh thiếu sáng (gamma correction), méo phối cảnh nhẹ (IoU từ 0,68–0,95).



Sai (FP): Logo hình tròn đồng tâm, mã Data Matrix có cấu trúc lưới tương tự QR.

Sai (FN): QR quá nhỏ (12×12 pixel) trên nền nhiễu, QR bị che khuất một finder pattern.

4 Phân tích và thảo luận

4.1 Khó khăn gặp phải

• Ảnh mờ, nhiễu, độ phân giải thấp → module QR bị đứt gãy, sai contour.

• Biến dạng phối cảnh mạnh (>45°) → tam giác vuông cân lệch, nội suy góc thứ tư sai.

• Che khuất một hoặc hai finder pattern → không đủ pattern để ghép QR.

• Họa tiết giống QR (logo tròn đồng tâm, Data Matrix) gây FP.

4.2 Hạn chế của phương pháp

• Phụ thuộc nhiều vào finder pattern; nếu cả ba bị che hoặc biến dạng quá mức → không phát hiện được.

• Thời gian xử lý tăng tuyến tính với số lượng QR (>15 QR/ảnh → >1,2 giây).

• IoU thấp (0,55–0,65) với QR méo phối cảnh mạnh.

• Chưa phân biệt được QR với Data Matrix.

4.3 Hướng cải thiện

• Phát hiện dựa trên lưới module (Hough lines) để không phụ thuộc hoàn toàn vào finder pattern.

• Bổ sung bộ lọc phân biệt QR / Data Matrix dựa trên viền đen liên tục.

• Tối ưu tốc độ bằng downsampling thông minh: xử lý thô ở tỷ lệ thấp, chỉ refine trên ROI.

• Áp dụng học máy nhẹ (Random Forest) trên các đặc trưng thủ công để giảm FP.

• Thay affine bằng homography để ước lượng góc thứ tư chính xác hơn khi méo phối cảnh.

5. Kết luận

Hệ thống phát hiện QR code dựa trên xử lý ảnh truyền thống đạt F1 số lượng 90,0% và F1 IoU 72%, tốc độ 0,5 giây/ảnh. Các kỹ thuật chính gồm phát hiện finder pattern, nhóm ba pattern bằng hình học tam giác vuông cân, tinh chỉnh bounding box bằng quét projection, và nhiều fallback cho trường hợp khó. Hạn chế chính là dễ bị ảnh hưởng bởi che khuất, biến dạng mạnh và nhầm lẫn với Data Matrix. Các hướng cải thiện đề xuất có thể nâng F1 lên trên 95% và giảm thời gian xử lý xuống dưới 0,25 giây/ảnh.